# Welcome to LightFM's documentation!

LightFM is a Python implementation of a number of popular recommendation algorithms for both implicit and explicit feedback.

It also makes it possible to incorporate both item and user metadata into the traditional matrix factorization algorithms. It represents each user and item as the sum of the latent representations of their features, thus allowing recommendations to generalise to new items (via item features) and to new users (via user features).

The details of the approach are described in the LightFM paper, available on arXiv.

## Quickstart

Jump straight to the Movielens quickstart if you're impatient.

## Installation

### PyPI

Install from pypi using pip: `pip install lightfm`. Everything should work out-of-the box on Linux, OSX using Homebrew Python, and Windows using Miniconda.

Note for OSX and Windows users: LightFM will by default not use OpenMP on OSX and Windows, and so all model fitting will be single-threaded. This is due to the fact that Clang (and Miniconda) does not support OpenMP, and installing an OpenMP-enabled version of gcc is complicated and labour-intensive. If you'd like to use the multi-threading capabilities of LightFM on these platforms, you should try using it via Docker as described in the next secion.

Building with the default Python distribution included in OSX is also not supported; please try the version from Homebrew or Anaconda.

### Using with Docker

On many systems it may be more convenient to try LightFM out in a Docker container. This repository provides a small Dockerfile sufficient to run LightFM and its examples. To run it:

1. Install Docker and start the docker deamon/virtual machine.
2. Clone this repository and navigate to it: `git clone git@github.com:lyst/lightfm.git && cd lightfm`.
3. Run `docker-compose build lightfm` to build the container.

The container should now be ready for use. You can then:

1. Run tests by running `docker-compose run lightfm py.test -x lightfm/tests/`
2. Run the movielens example by running `docker-compose run --service-ports lightfm jupyter notebook lightfm/examples/movielens/example.ipynb --allow-root --ip="0.0.0.0" --port=8888 --no-browser`. The notebook will be accessible at port 8888 of your container's IP address.

## Usage

Model fitting is very straightforward using the main LightFM class.

Create a model instance with the desired latent dimensionality:

```
from lightfm import LightFM

model = LightFM(no_components=30)
```

Assuming `train` is a (no_users, no_items) sparse matrix (with 1s denoting positive, and -1s negative interactions), you can fit a traditional matrix factorization model by calling:

```
model.fit(train, epochs=20)
```

This will train a traditional MF model, as no user or item features have been supplied.

To get predictions, call `model.predict`:

```
predictions = model.predict(test_user_ids, test_item_ids)
```

User and item features can be incorporated into training by passing them into the `fit` method. Assuming `user_features` is a (no_users, no_user_features) sparse matrix (and similarly for `item_features`), you can call:

```
model.fit(train,
          user_features=user_features,
          item_features=item_features,
          epochs=20)
predictions = model.predict(test_user_ids,
                            test_item_ids,
                            user_features=user_features,
                            item_features=item_features)
```

to train the model and obtain predictions.

Both training and prediction can employ multiple cores for speed:

```
model.fit(train, epochs=20, num_threads=4)
predictions = model.predict(test_user_ids, test_item_ids, num_threads=4)
```

This implementation uses asynchronous stochastic gradient descent [6] for training. This can lead to lower accuracy when the interaction matrix (or the feature matrices) are very dense and a large number of threads is used. In practice, however, training on a sparse dataset with 20 threads does not lead to a measurable loss of accuracy.

In an implicit feedback setting, the BPR, WARP, or k-OS WARP loss functions can be used. If `train` is a sparse matrix with positive entries representing positive interactions, the model can be trained as follows:

```
model = LightFM(no_components=30, loss='warp')
model.fit(train, epochs=20)
```

# Examples

Check the `examples` directory for more examples.

The Movielens example shows how to use LightFM on the Movielens dataset, both with and without using movie metadata. Another example compares the performance of the adagrad and adadelta learning schedules.

The Kaggle coupon purchase prediction example applies LightFM to predicting coupon purchases.

## Articles and tutorials on using LightFM

1. Learning to Rank Sketchfab Models with LightFM
2. Metadata Embeddings for User and Item Cold-start Recommendations
3. Recommendation Systems - Learn Python for Data Science

## How to cite

Please cite LightFM if it helps your research. You can use the following BibTeX entry.:

```
@inproceedings{DBLP:conf/recsys/Kula15,
  author    = {Maciej Kula},
  editor    = {Toine Bogers and
               Marijn Koolen},
  title     = {Metadata Embeddings for User and Item Cold-start Recommendations},
  booktitle = {Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender
               Systems co-located with 9th {ACM} Conference on Recommender Systems
               (RecSys 2015), Vienna, Austria, September 16-20, 2015.},
  series    = {{CEUR} Workshop Proceedings},
  volume    = {1448},
  pages     = {14--21},
  publisher = {CEUR-WS.org},
  year      = {2015},
  url       = {http://ceur-ws.org/Vol-1448/paper4.pdf},
}
```

## Development

Pull requests are welcome. To install for development:

1. Clone the repository: `git clone git@github.com:lyst/lightfm.git`
2. Install it for development using pip: `cd lightfm && pip install -e .`
3. You can run tests by running `python setup.py test`.

When making changes to the `.pyx` extension files, you'll need to run `python setup.py cythonize` in order to produce the extension `.c` files before running `pip install -e .`.